

U1-AT2 PYTHON CODE

1. PYTHON CODE:

Hospital Shift Scheduling using Backtracking

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
assignment = {}
```

Constraint checking function

```
def is_valid(doctor, shift):
```

```
    # Only one doctor per shift
```

```
    if shift in assignment.values():
```

```
        print(" ", shift, "already assigned")
```

```
        return False
```

```
    # D1 cannot work Night
```

```
    if doctor == "D1" and shift == "Night":
```

```
        print(" Constraint Failed : D1 cannot work Night")
```

```
        return False
```

```
    # D3 cannot work Morning
```

```
    if doctor == "D3" and shift == "Morning":
```

```
        print(" Constraint Failed : D3 cannot work Morning")
```

```
        return False
```

```
    # Check D2 before D3
```

```
temp = assignment.copy()
temp[doctor] = shift

if "D2" in temp and "D3" in temp:
    order = {"Morning":1, "Afternoon":2, "Night":3}

    if order[temp["D2"]] >= order[temp["D3"]]:
        print(" Constraint Failed : D2 must work before D3")
        return False

return True
```

Backtracking Function

```
def backtrack(index):

    if index == len(doctors):
        return True

    doctor = doctors[index]

    for shift in shifts:

        print("\nTrying", doctor, "->", shift)

        if is_valid(doctor, shift):

            assignment[doctor] = shift
```

```

        print(" Assigned:", assignment)

    if backtrack(index + 1):
        return True

    print(" Backtracking from", doctor)

    del assignment[doctor]

return False


print("Hospital Shift Scheduling\n")

if backtrack(0):

    print("\nFinal Valid Schedule\n")

    for d in doctors:
        print(d, "->", assignment[d])

else:
    print("No valid assignment found.")

```

2. **PYTHON CODE:**

```

from collections import deque

# Grid (S = Start, G = Goal, X = Obstacle)
grid = [

```

```
['S', '.', '.', 'X', '.'],  
['.', 'X', '.', 'X', '.'],  
['.', 'X', '.', '.', '.'],  
['.', '.', 'X', 'X', '.'],  
['.', '.', '.', 'G', '.']  
]
```

```
rows = cols = 5
```

```
start = (0,0)
```

```
goal = (4,3)
```

```
moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
queue = deque()
```

```
queue.append((start,[start]))
```

```
visited = {start}
```

```
print("Initial Queue :", list(queue))
```

```
while queue:
```

```
    (x,y), path = queue.popleft()
```

```
    print("\nExpanding Node :", (x,y))
```

```
    h = abs(x-goal[0]) + abs(y-goal[1])
```

```
    print("Manhattan Distance =", h)
```

```

if (x,y) == goal:
    print("\nGoal Reached!")
    print("Optimal Path :", path)
    print("Total Cost :", len(path)-1)
    break

for dx,dy in moves:

    nx = x + dx
    ny = y + dy

    if 0<=nx<rows and 0<=ny<cols:

        if grid[nx][ny] != 'X' and (nx,ny) not in visited:

            visited.add((nx,ny))
            queue.append(((nx,ny), path+[(nx,ny)]))

    print("Queue :", [node[0] for node in queue])

```

3. **PYTHON CODE:**

```
import heapq
```

```
# Grid
```

```
grid = [
    ['S', '.', '.', 'X', '.'],
    ['.', 'R', '.', 'X', '.'],
    ['.', 'X', '.', '.', '.'],
    ['.', '.', 'R', 'X', '.'],

```

```
['.', '.', '.', 'G', '.']  
]
```

```
rows = cols = 5
```

```
start = (0,0)
```

```
goal = (4,3)
```

```
moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
pq = []
```

```
heapq.heappush(pq,(0,start,[start]))
```

```
visited = {}
```

```
print("Starting Rescue Robot...\n")
```

```
while pq:
```

```
    cost,(x,y),path = heapq.heappop(pq)
```

```
    if (x,y) in visited and visited[(x,y)] <= cost:
```

```
        continue
```

```
    visited[(x,y)] = cost
```

```
    print("Robot at :", (x,y))
```

```
    print("Current Cost :", cost)
```

```
if (x,y)==goal:
```

```
    print("\nSurvivor Found!")
```

```
    print("Optimal Path :", path)
```

```
    print("Minimum Cost :", cost)
```

```
    break
```

```
print("Adjacent Cells:")
```

```
for dx,dy in moves:
```

```
    nx = x + dx
```

```
    ny = y + dy
```

```
if 0<=nx<rows and 0<=ny<cols:
```

```
    if grid[nx][ny] != 'X':
```

```
        new_cost = cost + 1
```

```
    if grid[nx][ny] == 'R':
```

```
        new_cost += 2
```

```
    print((nx,ny),"Discovered")
```

```
    heapq.heappush(
```

```
        pq,
```

```
        (new_cost,(nx,ny),path+[(nx,ny)])
```

```
    )
```

```
print("Knowledge Updated\n")
```